

# THE PASCAL FRONTIER

MODERN AI INFERENCE ON HARDWARE  
THAT DIDN'T GET THE RETIREMENT MEMO

WHITEPAPER 02

## IT'S ALIVE.

SGLang on NVIDIA V100 / SM70

Bringing up a 35B hybrid MoE—and finding its serving boundary

**J. SCOTT HENDERSON**

Mostly for engineers. Translated for curious humans.

Technical graphic whitepaper · September 2026

Evidence window: August 9–10, 2026 · Candidate for Founder review

01

# Same machine. New engine.

What WP1 established—and the question WP2 answers

WP1 established that Lurch could serve modern models through selected vLLM and llama.cpp paths. WP2 follows the same V100 into SGLang, where fitting the weights was only the beginning.

## THE INHERITANCE

The published first paper supplies the platform dossier, earlier compatibility history, and the rule for comparisons: match the workload before declaring a winner. WP2 does not revise those results.

Read The Pascal Frontier — Whitepaper #1  
Useful entry points: platform dossier p. 6; comparison contract pp. 13–14; results pp. 16–18; series handoff p. 23.

## THE NEW FINDING

The 35B-class hybrid MoE reached model loading, hybrid cache initialization, CUDA graph capture, startup, and HTTP inference after a documented four-commit patch sequence.

The later common-harness run completed 1 user × 2 requests. Under the tested configuration, 2 × 5 and 4 × 5 failed with CUDA/Triton memory exhaustion.

### 01 Build the bridge

Align architecture, weight format, cache behavior, and output contract.

### 02 Prove it answers

Separate a working API from a sustained serving qualification.

### 03 Keep the failures

Use the concurrency boundary to define the conclusion.



LURCH / FIELD NOTES

## Loading the model was the opening act.

### FOR THE REST OF US

*WP1 tested earlier engines in this machine. WP2 installs a different “transmission” and asks both whether it moves and what happens when traffic arrives.*

Sources: S01, S02, S04–S06. Interpretation is specific to the preserved configuration.

02

Table of contents

Every section. Exact pages. Direct navigation.

THE INVESTIGATION

|                                        |    |
|----------------------------------------|----|
| Same machine. New engine.              | 02 |
| Humans, AI, and authority              | 04 |
| Lurch, in this experiment              | 05 |
| From qualification to the 35B boundary | 06 |

ENGINEERING DEEP DIVE

|                                        |    |
|----------------------------------------|----|
| Four commits. One working contract.    | 07 |
| Two files. Two responsibilities.       | 08 |
| Forty layers. Two kinds of work.       | 09 |
| Names are part of the interface        | 10 |
| Packed weights are not ordinary arrays | 11 |
| The answer must leave the model        | 12 |

RESULTS

|                                            |    |
|--------------------------------------------|----|
| The server answered.                       | 13 |
| Two requests passed. Ten did not.          | 14 |
| The failure happened on the GPU path       | 15 |
| Cool does not automatically mean efficient | 16 |

INTERPRETATION

|                                         |    |
|-----------------------------------------|----|
| What WP1 lets us compare                | 17 |
| Compatibility earned. Capacity bounded. | 18 |

APPENDICES & HANDOFF

|                                              |    |
|----------------------------------------------|----|
| Reproduce the contract, not just the command | 19 |
| Claim audit before publication               | 20 |
| Source map and remaining gaps                | 21 |
| Next: Save vLLM. Run the 35B.                | 22 |

HOW TO READ THE EVIDENCE

**OBSERVED** — preserved run or transcript. **DOCUMENTED** — engineering record; independent verification may remain open. **ANALYSIS** — interpretation of evidence. **HYPOTHESIS** — unisolated explanation. **GAP / SUPERSEDED** — incomplete proof or replaced status.

**Evidence hierarchy:** raw records → engineering record → review draft → promotional claim. Later direct evidence can correct an older summary; successful startup does not establish concurrent serving capacity.

FOR THE REST OF US

Start with the question on page 2. Follow the engineering on pages 7–12, then check the 35B results on pages 13–16. The audit and source map show what supports the conclusions.

03

Humans, AI, and authority

Who did the work—and what this edition verifies

J. Scott Henderson performed the engineering and operated the hardware. This edition reconciles the preserved record; it does not rerun the experiment.

| Role / phase           | Responsibility and boundary                                                                                                                                                                |
|------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Original research      | Scott ran commands and benchmarks, investigated failures, and made engineering decisions. Ren assisted research, debugging analysis, documentation, and editorial development.             |
| Current evidence audit | Codex/Ren extracted archived records, checked manifest hashes, reconciled claims, inspected benchmark utilities, and checked available GitHub records.                                     |
| Current production     | AI-assisted writing and layout; deterministic document generation and PDF rendering. The selected model label is not independently recorded in the production evidence and is not guessed. |
| Artwork                | Previously produced Pascal Frontier narrative illustrations are reused with provenance. They depict the platform but do not establish hardware state, temperature, or performance.         |
| Final authority        | Scott reviews the publication candidate and authorizes any release. Creating this package does not publish it.                                                                             |

WHAT IS VERIFIED HERE

Archive integrity checks and consistency among client, summary, run, and telemetry records. The 35B result is evidence-backed historical behavior, not an independent replication.

PATCH PROVENANCE

The four commits form a verified parent-linked sequence in the SGLang checkout on Lurch. The final commit matches the archived 35B benchmark metadata. Code provenance supports the engineering account; the benchmark records establish the measured behavior.



LURCH / FIELD NOTES

The logs get a vote.  
The human makes the call.

FOR THE REST OF US

Think of this as auditing the flight recorder, not flying the aircraft again. It can establish what was recorded while still leaving replication and publication approval separate.

04

Lurch, in this experiment

A historical launch envelope, not today's machine inventory



Figure 1. Narrative representation carried forward from WP1. The visible controller display is artwork, not run telemetry.

| Item                  | Recorded WP2 value                                                      |
|-----------------------|-------------------------------------------------------------------------|
| GPU                   | Tesla PG500-216 / V100-class; 32,768 MiB; SM70 / compute capability 7.0 |
| Host baseline         | AMD Ryzen 5 3600; 64 GB RAM                                             |
| Driver                | 580.173.02                                                              |
| Framework             | PyTorch 2.9.1+cu128 in the archived benchmark records                   |
| SGLang version string | 0.0.0.dev13451+ga5929c72a                                               |
| 35B code identity     | 3231ec8a4e63f738486c6f013532327bda61624e                                |
| CUDA lane             | 12.8; TORCH_CUDA_ARCH_LIST=7.0                                          |
| Attention / context   | flash_attn_v100; 4,096 tokens                                           |
| Memory controls       | mem_fraction_static=0.90; SM70 JIT headroom=2.0 GB                      |

VERSION STRING ≠ COMPLETE CODE IDENTITY

The installed SGLang version string still names the base lineage while the 35B run manifest records the later wrapper commit. Cite both; using the package version alone loses the patch identity.

WP1 and the canonical ANTON-AOS hardware record establish the broader platform context. This table pins the historical SGLang lane. It does not borrow a toolkit version or attention backend from the earlier vLLM experiments.

FOR THE REST OF US

The machine is familiar, but the software lane has its own parts list. A driver's advertised CUDA compatibility is not the same as the toolkit used to build this experiment.

05

# From qualification to the 35B boundary

What the SGLang runs establish

SGLang completed the selected 14B and 32B workloads. The 35B hybrid MoE then exposed a different challenge: integrating its architecture and weight format, followed by serving concurrent requests.

| Model / stage                | Recorded workload result               | Engineering significance                                                   |
|------------------------------|----------------------------------------|----------------------------------------------------------------------------|
| 14B GPTQ INT4, early attempt | Server startup failure                 | This configuration did not reach serving readiness.                        |
| 14B later configuration      | 8 users × 10 requests; 80/80 completed | The selected SGLang configuration completed the client workload.           |
| 32B GGUF                     | 8 users × 10 requests; 80/80 completed | The selected quantized configuration also completed the workload.          |
| 35B hybrid MoE GGUF          | 1×2 PASS; 2×5 and 4×5 FAIL             | Model compatibility was achieved; concurrent serving remained constrained. |

01

### Qualify

Establish whether each specific configuration completes its client workload.

02

### Integrate

Align the 35B hybrid architecture, GGUF weights, cache, and output contract.

03

### Measure the limit

Separate successful 35B inference from the failed concurrent workloads.

## COMPLETION IS A BOUNDED RESULT

The 14B and 32B runs supply qualification context. Their completion counts do not predict 35B capacity, establish output quality, or support a throughput ranking across different configurations. The following sections explain the 35B integration and its measured serving boundary.

FOR THE REST OF US

Passing a test with one engine configuration does not certify every other configuration. The larger hybrid model needed its own integration work and its own load test.

Sources: S07–S09 (14B/32B); S04–S06 (35B). The companion audit records source integrity checks.

06

# Four commits. One working contract.

The documented patch sequence and its responsibilities

The bring-up crossed several subsystem boundaries. The cleaned patch series separates those responsibilities into four reviewable changes.

| Commit    | Documented subject and responsibility                                                                                                                                           |
|-----------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 35b22351d | <b>feat(qwen3.5): add MoE text hybrid runtime support</b><br>Config registration/export, preserved layer layout, hybrid GDN recognition, and Mamba/radix handling.              |
| 4d6214251 | <b>feat(gguf): add Qwen3.5 MoE config and tensor mapping</b><br>Explicit HF configuration for GGUF weights; architecture and hybrid tensor-name mapping.                        |
| 4cebf0740 | <b>fix(gguf): support Qwen3.5 MoE packed quantized loading</b><br>Packed loaders, scalar metadata, namespace/fused-name handling, singleton shapes, and expert materialization. |
| 3231ec8a4 | <b>feat(qwen3.5): add standalone MoE causal LM wrapper</b><br>Reusable body, LM head, LogitsProcessor, and standalone entry point.                                              |

```
Recorded base: a5929c72ac2563e699017d30faa52e02b4793af1
Recorded branch: experiment/qwen35moe-gguf-config-bypass
Archived 35B run HEAD:
3231ec8a4e63f738486c6f013532327bda61624e
```

## A TRACEABLE PATCH SEQUENCE

The Git parent chain connects all four commits to the recorded base. The final commit matches the 35B benchmark manifest. Together, the commits change nine files: configuration ownership, GGUF loading, hybrid runtime integration, and the standalone causal-LM output path.

FOR THE REST OF US

*The repairs are grouped like separate workshop jobs: identify the machine, read its parts, install them correctly, and connect the output. That makes the engineering easier to review and maintain.*

## 07

# Two files. Two responsibilities.

GGUF weights, HF configuration, and config-class ownership

## GGUF checkpoint

Quantized weights and tensor names

+

## Explicit HF config

Architecture and layer semantics



## SGLang-owned configuration class

Register / export → preserve layer\_types → recognize hybrid capabilities

### THE FAILURE

Deriving the required architecture from the GGUF path alone was insufficient in the documented bring-up. HF and SGLang Qwen3.5 MoE text configuration classes were distinct objects, so runtime checks could miss the configuration they received.

### THE REPAIR

Keep GGUF as the weight source, supply an explicit native HF configuration through the existing configuration-file flow, register SGLang configuration classes in Transformers CONFIG\_MAPPING, and export Qwen3\_5MoeTextConfig.

```
--load-format gguf
--model-config-parser hf
--decrypted-config-file "$CONFIG"
--tokenizer-path Qwen/Qwen3.6-35B-A3B
```

The target artifact is Hermes3.6-35B-A3B-Uncensored-Genesis-V7-APEX.gguf. The recorded SGLang configuration model type is qwen3\_5\_moe\_text. Similar family names do not eliminate the need to align the actual class and field contracts.

#### FOR THE REST OF US

*A crate of parts does not replace the assembly drawing. GGUF supplies the weights; configuration tells this runtime which machine those weights belong to.*

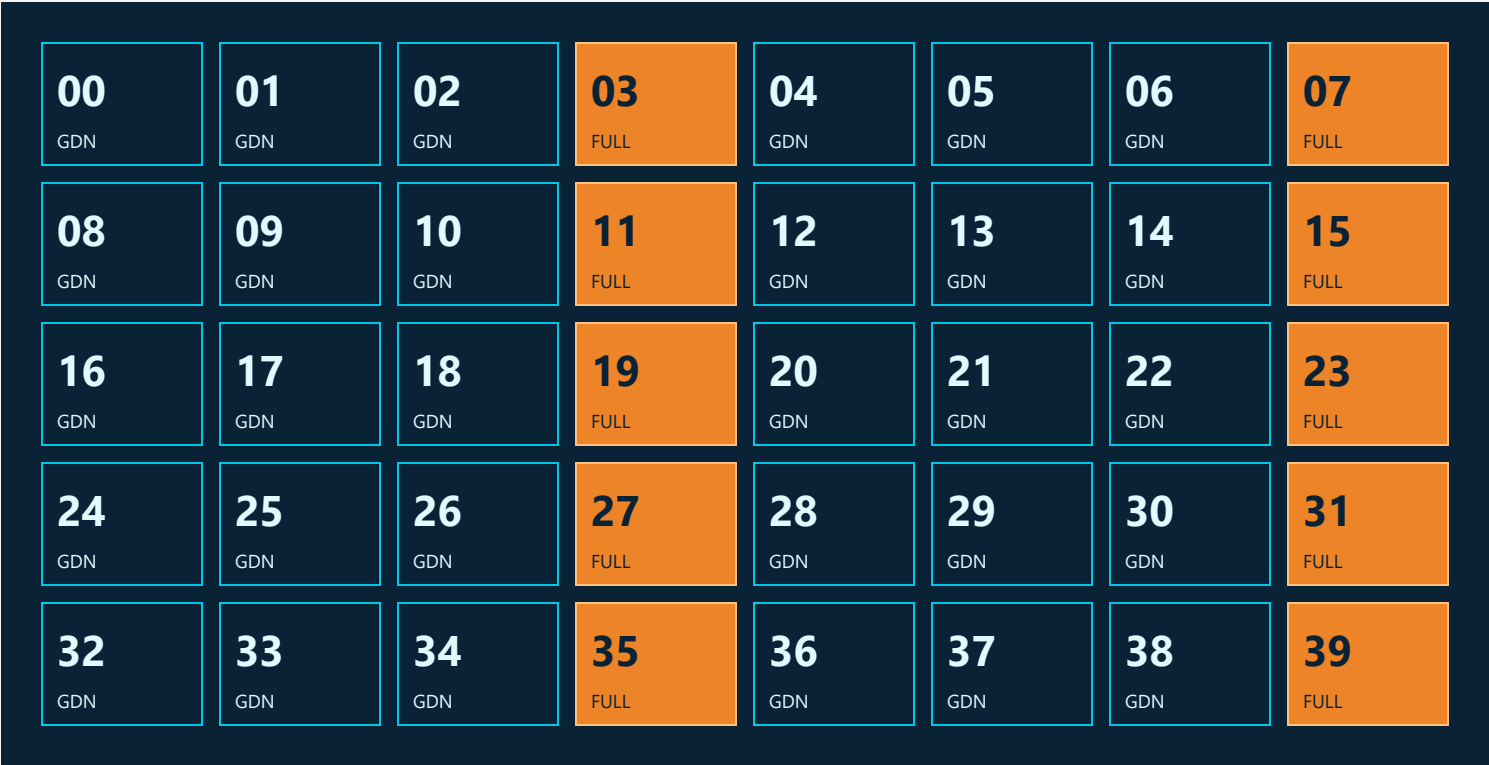
Sources: S02 §§4.3–5, S03, S04 command.txt/run.yaml. Figure 2 is an explanatory contract diagram.

08

Forty layers. Two kinds of work.

Preserving the hybrid layout instead of flattening its meaning

The recorded 35B configuration contains 30 linear-attention/GDN layers and 10 full-attention layers. Treating it as a uniform stack loses the architecture that the caches and kernels must serve.



30 GDN / linear-attention

10 full-attention

Zero-based layer IDs

Full-attention IDs:  
3, 7, 11, 15, 19, 23, 27, 31, 35, 39

Preserve: layer\_types

Recognize: hybrid GDN / Mamba state requirements

WHY PRESERVATION MATTERS

The engineering record identifies a loss of checkpoint layer layout during SGLang configuration derivation. Retaining layer\_types and recognizing the resulting hybrid structure keeps downstream behavior aligned with the model.

WHAT THE DIAGRAM DOES NOT CLAIM

This is the documented layout, not a measured timing profile. Equal-sized cells do not imply equal compute, memory, parameter count, or per-layer latency.

FOR THE REST OF US

This building alternates ordinary rooms with rooms that need different services. The floor plan has to survive the handoff; one generic room label will not do.

## 09

# Names are part of the interface

Tensor mapping, prefixes, and already-fused parameters

## 01 Architecture translation

Map qwen3\_5\_moe\_text and the recorded Qwen3.5/Qwen3.6 family handling to GGUF qwen35moe.

## 02 Hybrid tensor mapping

Map linear-attention names explicitly; retain dedicated handling for dt\_bias.

## 03 Destination namespace

Reconcile model.\* names emitted by mapping with the body's flat layers.\*, embed\_tokens.\*, and norm.\* names.

## 04 Fused names

Recognize a target that is already fused before applying stacking rules again.

### NAMESPACE MISMATCH

The body/expert loading path normalizes a leading model. prefix where the registered destination is flat. This is a targeted correspondence rule, not permission to strip arbitrary path components.

### DOUBLE FUSION

An already-fused name can be stacked again into an invalid duplicated qkv-style name. The recorded fix skips stacking when the target fused parameter is already present.

**Source name → explicit mapping → registered parameter**

Mapping errors can look like missing tensors even when the bytes are present. A maintainable loader needs a testable map between the checkpoint representation and the runtime's registered parameters, including the exceptional hybrid names.

#### FOR THE REST OF US

*The parcel arrived, but the address used the wrong building entrance—and another label tried to bundle an already bundled delivery. Fix the address and handling rules, not the contents.*

Sources: S02 \$5, S03. Diagram and explanation are analysis of the documented fixes.

## 10

# Packed weights are not ordinary arrays

Quantization metadata, singleton dimensions, and expert lifecycle

## PACKED LOADER BINDING

The merged GDN modules needed loader bindings for `qweight` and `qweight_type`. Treating packed representations as ordinary dense weights breaks the load contract.

## SCALAR METADATA

The quantization-type destination can be zero-dimensional. Preserve scalar shape for `qweight_type` rather than forcing a vector into a scalar parameter.

### Packed tensors

`qweight + qweight_type`



### Validate the destination

`Loader binding · metadata rank · names · permitted singleton shape`



### Materialize experts after loading

`Initialize fused_experts=None; process_weights_after_loading()`

## SHAPE REPAIR WITH A GUARD

Restore singleton dimensions only when element counts match and the loaded shape equals the registered parameter shape with size-1 dimensions removed. This repairs the documented squeezed gate/conv cases without licensing an arbitrary reshape.

The GGUF MoE path also needed lifecycle initialization and post-load expert-weight materialization. Successful deserialization is not the end of preparation for inference.

### FOR THE REST OF US

*Compressed parts need the matching unpacker and an assembly step. Adding a missing spacer is safe only after proving the rest of the part already matches.*

Sources: S02 §5; S03. Figure 4: documented packed-weight loading lifecycle; explanatory schematic.

## 11

# The answer must leave the model

Hybrid cache integration and a real causal-LM output contract

## STATE BELONGS IN THE CACHE PATH

The standalone Qwen3.5 MoE causal architecture had to be recognized by the existing hybrid Mamba server-argument paths. The documented repair reused the established Mamba radix-cache handling; the successful runtime reports MambaRadixCache and hybrid\_ssm=True.

## HIDDEN STATES ARE NOT LOGITS

The old standalone path returned a hidden-state Tensor. ModelRunner expected a LogitsProcessorOutput exposing next\_token\_logits. An object can contain valid computations and still violate its caller's interface.

Qwen3\_5MoeModel

Reusable model body



ParallelLMHead

Project hidden states toward vocabulary scores



LogitsProcessor

Provide the expected serving output



next\_token\_logits

Contract consumed by the runner

The engineering record describes splitting Qwen3\_5MoeModel from a genuine standalone Qwen3\_5MoeForCausalLM wrapper, retaining the multimodal body path and exposing the standalone entry class. This is a separation of responsibilities, not simply a renamed return value.

### FOR THE REST OF US

*A kitchen can prepare the meal correctly and still fail service if it sends ingredients to the dining room instead of a plate. The wrapper is responsible for delivering what the next component expects.*

Sources: S02 §§5–8; S04 server.log. Figure 5: output-contract schematic, not a performance trace.

12

# The server answered.

A compatibility milestone, with two capture sessions kept separate

| Milestone                            | Preserved evidence                                                                                                                              |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------|
| Final bring-up in engineering record | CUDA graph batches 1, 2, 4, 8, 12, 16, 18; capture 1.80 s; available memory 0.98 GB.                                                            |
| Later benchmark session              | Run 20260810-181949: graph capture 13.96 s; available memory 0.98 GB. These are different sessions.                                             |
| Hybrid state                         | MambaRadixCache; hybrid_ssm=True.                                                                                                               |
| Startup and API                      | Application startup; /model_info HTTP 200; /generate HTTP 200. The benchmark also records successful /v1/completions responses.                 |
| Reported capacity fields             | max_total_num_tokens=100120; max_running_requests=18. These are runtime configuration/capacity reports, not a demonstrated 18-client benchmark. |

```
Capture cuda graph end. Time elapsed: 13.96 s.  
mem usage=0.21 GB. avail mem=0.98 GB.  
  
GET /model_info HTTP/1.1      200 OK  
POST /generate HTTP/1.1      200 OK  
POST /v1/completions HTTP/1.1 200 OK
```

Excerpted from the later successful benchmark server log; prefixes and local request addresses omitted for readability/privacy. Values unchanged.



LURCH / FIELD NOTES

# Ready to roll. Now measure the road.

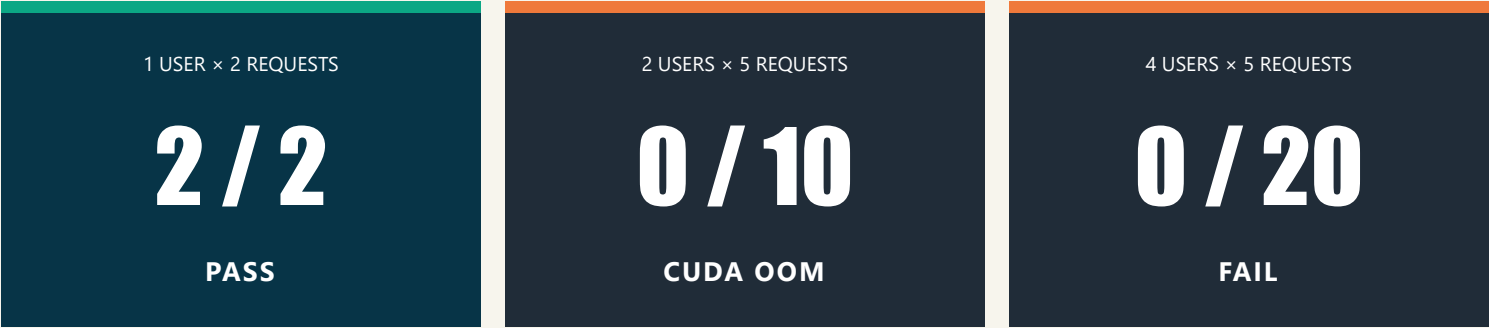
FOR THE REST OF US

Starting an engine, answering a request, and carrying traffic are separate achievements. A dashboard capacity setting is not a road test with that many passengers.

13

Two requests passed. Ten did not.

The standardized 35B harness boundary



| 1×2 metric                | Recorded value                | Interpretation                                                 |
|---------------------------|-------------------------------|----------------------------------------------------------------|
| Elapsed request workload  | 33.8610 s                     | Client workload duration; excludes earlier startup.            |
| Input / output tokens     | 60 / 64                       | Counts recorded by the harness; 124 combined.                  |
| Input / output throughput | 1.77 / 1.89 tokens/s          | Distinct quantities; neither equals total throughput.          |
| Total throughput          | 3.66 tokens/s                 | (60 + 64) / 33.8610; not output-only decode speed.             |
| p50 / p95 / p99 latency   | 16.9298 / 31.7375 / 33.0538 s | Request end-to-end latencies, not TTFT; only two observations. |

THE WORKLOAD WAS SMALL AND NON-RANDOMIZED

The preserved command uses 4,096 context, 32 maximum output tokens, zero think time, and explicit user/request counts. random\_stress=false. Randomized range fields retained in run.yaml are not evidence that those ranges were exercised.

The observed pass proves this two-request workload completed. It is not evidence of sustained reliability, long-context operation, answer quality, or a statistically stable latency distribution. Both failed concurrency lanes preserve disconnects followed by refused connections.

FOR THE REST OF US

One customer could finish two orders. When two or four customers arrived together, the service failed. That is a useful boundary—even though the successful sample is far too small for a broad capacity promise.

Sources: S04–S06 client.log, command.txt, run.yaml; S13 client implementation. Figure 6: completion counts, not a throughput comparison.

## 14

# The failure happened on the GPU path

Concurrency, Triton CUDA OOM, and the limits of diagnosis

## 01 Readiness first

The later server reaches readiness and responds to model discovery.

## 02 Concurrent work begins

First simultaneous requests disconnect: two in 2×5; four in 4×5.

## 03 Server failure

The request-time server trace reports Triton Error [CUDA]: out of memory.

## 04 Aftermath

Subsequent requests receive connection refused; the client records no completed requests.

```
RuntimeError: Triton Error [CUDA]: out of memory
```

```
CLIENT_ERROR: ... Remote end closed connection without response
```

```
CLIENT_ERROR: ... [Errno 111] Connection refused
```

```
2×5: total=10 | completed=0 | failures=10
```

```
4×5: total=20 | completed=0 | failures=20
```

## OBSERVED

The CUDA allocation failure occurs on the serving path. The failing workload has a concrete archive ID and request record. Earlier startup traces also contain OOM text, including in the later PASS log; an unqualified keyword count would misclassify the run.

## ANALYSIS / HYPOTHESIS

Weights, caches, captured graphs, and transient workspaces compete for memory. The record supports an allocation boundary, but does not isolate a single allocation as the sole cause or prove a universally safe client limit.

The review draft explains the later shell “Killed” as cleanup after child failure and reports no host-OOM entry in dmesg. The authoritative finding here is narrower: the preserved server log directly records CUDA/Triton OOM. A host-OOM exclusion is not independently established without the corresponding kernel log.

### FOR THE REST OF US

*A sign saying “18 seats” does not mean the kitchen can handle 18 simultaneous orders. The working set during service can exceed what startup needed.*

Sources: S05–S06 request-time server/client traces; S03 for the historical host-cleanup explanation.

15

Cool does not automatically mean efficient

Power and temperature within the preserved monitoring window

| 35B run      | Status | Peak °C | Average W | Peak W |
|--------------|--------|---------|-----------|--------|
| 1×2 / 181949 | PASS   | 37.0    | 40.2      | 101.5  |
| 2×5 / 183326 | FAIL   | 37.0    | 41.3      | 121.8  |
| 4×5 / 182907 | FAIL   | 37.0    | 40.5      | 104.8  |



Figure 7. Observed peak GPU power; common 0–130 W scale. Failed runs remain visibly classified.

MEASUREMENT WINDOW

The harness starts nvidia-smi sampling before server startup and stops monitoring after the client/test sequence and cleanup. Reported averages therefore include startup/loading and other non-generation time; they are not generation-only averages.

WHAT WE DO NOT INFER

No tokens-per-joule result is calculated from a client-only duration multiplied by a whole-run average. No cross-engine efficiency winner, thermal steady state, or absence of throttling is inferred from these short runs.

The baseline for the passing 1×2 run is recorded as 25.00°C and 20.11 W. The telemetry is GPU-level, not whole-system wall power. The captured ~1-second sampling cadence can also miss shorter transients.

FOR THE REST OF US

A restaurant that closed halfway through dinner may have used little electricity. That does not prove it served meals efficiently. Compare useful completed work over the same measured time window.

16

# What WP1 lets us compare

Runtime behavior, without a manufactured leaderboard

WP1’s llama.cpp evidence demonstrates that this hardware/model family is not categorically incapable of serving the 35B workload. It does not make every SGLang and llama.cpp number directly comparable.

| Question                                       | WP1 → WP2 connection                                                                                                          |
|------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|
| Can the hardware participate?                  | WP1 establishes earlier successful engine paths; WP2 demonstrates a patched SGLang path reaching inference.                   |
| Can the 35B artifact be served?                | The preserved llama.cpp run completed 80/80 client requests. SGLang’s preserved configuration passed 1×2 and failed 2×5/4×5.  |
| Is one engine faster?                          | Not established by juxtaposing unlike client workloads, context choices, output budgets, quantization paths, or warmup state. |
| Does client concurrency equal GPU parallelism? | No. Eight client streams do not establish eight independent simultaneous GPU execution paths.                                 |
| Is lower power better efficiency?              | Not without aligned measurement windows, successful work, and a defensible energy denominator.                                |



The cross-engine observation is qualitative and bounded: the same platform can exhibit materially different serving behavior across runtime configurations. Scheduler, cache, format, and temporary-memory design are candidate explanations, not isolated causal measurements.

FOR THE REST OF US

*The same car can behave differently with another transmission. To compare lap times, keep the route, load, fuel, and timing rules the same—not just the chassis.*

## 17

# Compatibility earned. Capacity bounded.

What the engineering supports—and what it does not

## CONCLUSION

The preserved experiment demonstrates that targeted integration work can bring this 35B hybrid MoE GGUF through SGLang on a single SM70 V100: configuration, loader, cache, graph, and output contracts all matter.

The observed serving envelope remains narrow: a two-request single-user pass, followed by failures at the tested 2×5 and 4×5 loads. A working API is a compatibility achievement, not a production capacity certification.

## MAINTAINABILITY ANALYSIS

The patch sequence exposes where model-family knowledge crosses boundaries. Explicit capability declarations, isolated format mapping, guarded shape conversions, and a clear body/wrapper split would make similar integrations easier to test and reason about.

These are architectural recommendations from one bring-up record, not a measured refactor or a claim that a general solution has been implemented.

### 01 Model adapter

Declare hybrid state, layer layout, MoE behavior, and the output contract.

### 02 Weight-format adapter

Own names, packed metadata, shapes, and post-load materialization.

### 03 Runtime / hardware

Consume those capabilities with explicit backend and memory boundaries.



LURCH / FIELD NOTES

# It ran. The boundary matters too.

### FOR THE REST OF US

*The result is neither “impossible” nor “unlimited.” The machine crossed the compatibility barrier, and the tests identified where the current service stopped coping.*

Sources: S02–S06. Conclusions are bounded synthesis; recommendations remain analysis.

18

# Reproduce the contract, not just the command

A historical launch recipe and the metadata it still needs

Paths below use portable placeholders. This is the documented bring-up envelope, not a promise that a fresh installation or an unpatched upstream checkout will behave identically.

```
export CUDA_HOME=/usr/local/cuda-12.8
export PATH="$CUDA_HOME/bin:$PATH"
export LD_LIBRARY_PATH="$CUDA_HOME/lib64:${LD_LIBRARY_PATH:-}"
export TORCH_CUDA_ARCH_LIST=7.0
export SGLANG_SM70_JIT_HEADROOM_GB=2.0

python -m sglang.launch_server \
  --model-path "$MODEL" \
  --tokenizer-path Qwen/Qwen3.6-35B-A3B \
  --load-format gguf --model-config-parser hf \
  --decrypted-config-file "$CONFIG" \
  --host 127.0.0.1 --port 30000 \
  --context-length 4096 \
  --attention-backend flash_attn_v100 \
  --mem-fraction-static 0.90
```

| Pin / preserve                       | Why it matters                                                                                                                    |
|--------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------|
| Source commit and patch series       | The installed version string does not fully identify the modified tree.                                                           |
| Model bytes, tokenizer, config bytes | A filename or model-family label is not a content hash. Exact model/config artifact hashes are not established in this candidate. |
| Harness and generated client         | The client implements workload shape, endpoints, time measurement, and percentile calculation.                                    |
| Run directory                        | Keep command, environment, client/server logs, telemetry, results, and checksums together.                                        |
| Monitoring window / state            | Record startup, warmup, request interval, cooldown, and residual VRAM separately.                                                 |

The benchmark archive preserves the actual run commands and generated clients. Its manifest flag `comparable_cross_runtime=true` is metadata, not proof that every historical run satisfies the comparison contract.

FOR THE REST OF US

*A recipe needs both instructions and exact ingredients. Preserving the command is necessary, but it cannot compensate for missing model hashes or a different software checkout.*

19

# Claim audit before publication

Decisions made while reconciling the evidence

| Claim / temptation                                      | Audit disposition                                                                                                              |
|---------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------|
| "14B/32B completion predicts 35B capacity"              | NOT ESTABLISHED. The selected 14B and 32B runs each completed 80/80 requests; 35B has its own measured serving boundary.       |
| "35B inference success establishes concurrent capacity" | REJECT. The 1×2 workload passed; the tested 2×5 and 4×5 workloads failed.                                                      |
| "35B captured graphs in 1.80 s"                         | DOCUMENTED for final bring-up. The later successful benchmark records 13.96 s; do not merge the sessions.                      |
| "18 requests supported"                                 | REJECT as demonstrated capacity. 18 is a runtime-reported field; tested 2×5/4×5 failed.                                        |
| "3.66 tokens/s decode speed"                            | CORRECT to total input+output throughput. Output-only rate is 1.89 tokens/s in the tiny passing run.                           |
| "Triton OOM proves host RAM exhaustion"                 | REJECT. The observed error is CUDA/Triton OOM. Kernel-log exclusion remains unverified here.                                   |
| "37°C proves excellent efficiency"                      | REJECT. Short monitoring windows and failed work cannot establish efficiency or steady-state thermals.                         |
| "Patch provenance establishes runtime performance"      | REJECT. All four commits are verified in the Lurch checkout; performance claims come from the corresponding benchmark records. |
| "All archive checksums match"                           | REJECT. One early GPTQ failed-run server.log mismatches; selected successful and 35B result sets pass.                         |

The accompanying claim ledger maps material claims to sources and page numbers. The source manifest pins reviewed files by SHA-256. Review drafts are editorial aids; raw evidence governs numerical results.

FOR THE REST OF US

*The audit removes claims that sound stronger than the evidence. A smaller, precise conclusion is more useful than a dramatic one that cannot survive a source check.*

20

Source map and remaining gaps

How to trace the paper back to its records

| ID        | Source and authority                                                                                  |
|-----------|-------------------------------------------------------------------------------------------------------|
| S01       | Published WP1, ProjectDominionStudios/Public; platform and series context.                            |
| S02       | PDS-Volta-SGLang-Engineering-Record-2026-08-10.docx; documented engineering chain and final bring-up. |
| S03       | The-Pascal-Frontier-WP02-SGLang-V100-REVIEW.docx; editorial draft, subordinate to raw evidence.       |
| S04       | 20260810-181949...u1_r2; 35B successful run, all 13 listed hashes pass.                               |
| S05       | 20260810-183326...u2_r5; 35B failure run, all 13 listed hashes pass.                                  |
| S06       | 20260810-182907...u4_r5; 35B failure run, all 13 listed hashes pass.                                  |
| S07       | 20260809-164028...14b...u8_r10; completed qualification, 13/13 hashes pass.                           |
| S08       | 20260809-171239...32b...u8_r10; completed qualification, 13/13 hashes pass.                           |
| S09       | 20260809-024916...14b...gptq-int4...u1_r2; early startup failure; one server-log hash mismatch.       |
| S10 / S13 | PDS benchmark v1.0.5 utility / generated concurrent-client.py; measurement semantics.                 |
| S11 / S12 | Studio publication/editorial standards / ANTON-AOS historical hardware record.                        |
| S14 / S15 | llama.cpp 20260807-203531 results / verified SGLang four-commit chain and patch diff.                 |

GAPS THAT REMAIN

Exact model/config content hashes; independent replay; steady-state energy and thermal characterization; broad concurrency/context sweeps; statistical latency confidence; and the early failed-run server-log integrity mismatch.

The companion source manifest links each claim to its supporting records. Experimental limits distinguish demonstrated behavior from the further measurements needed for broader conclusions.

FOR THE REST OF US

The source map is the route back to the receipts. Gaps identify what cannot yet be claimed; they do not erase the results that the measurements support.

21

# Next: Save vLLM. Run the 35B.

The Pascal Frontier — Whitepaper #3

THE PASCAL FRONTIER / WHITEPAPER 02

## SGLANG ENTERED. THE LOGS STAYED.

The 35B hybrid model reached inference.  
The concurrent workload found the limit.



WP1 established the platform and earlier engine paths. WP2 adds a documented SGLang integration and a measured failure boundary. Together, they show why hardware capability and software support must be investigated as separate questions.

**Next: The Pascal Frontier — Whitepaper #3.** Scott, Ren, and Lurch return to vLLM to investigate whether disciplined engineering can carry it across the 35B-class loading boundary. SGLang has answered its own compatibility question; the next chapter returns to the unresolved vLLM challenge. No victory is assumed.

### FOR THE REST OF US

*The same machine meets a different obstacle. WP3 returns to the engine that could not yet load the 35B-class model and asks whether the team can get it running.*

Series handoff: S01, WP1 p. 23. WP2 conclusion: S02–S06. No WP3 result is claimed.